“Disk Scheduling Algorithm Visualizer”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

SANKEERTHANA THANMAYE 2520090193
VIKRAM IMMELLA 2520090118

Under the Esteemed Guidance of

Dr. M. Raghupathi
[Designation]

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **"Disk Scheduling Algorithm Visualizer"** is a record of Bonafide work of **Sankeerthana Thanmaye – 2520090193, Vikram Immella – 2520090118** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The work reported here was carried out by us under the guidance of Dr. M. Raghupathi. The results embodied in this report have not been copied from any other departments/University/Institute.

SANKEERTHANA THANMAYE	**2520090193**
VIKRAM IMMELLA	**2520090118**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the project based report entitled **"Disk Scheduling Algorithm Visualizer"** is a bonafide work done and submitted by **Sankeerthana Thanmaye – 2520090193, Vikram Immella – 2520090118**, in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year 2026–27. The work was carried out under the guidance of **Dr. M. Raghupathi**, [Designation], Department of Computer Science and Engineering.

Signature of the Guide
(Dr. M. Raghupathi)

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance of many people. We, the team members, express our sincere thanks to all those who assisted us in one way or the other in the completion of this project.

We express our greatest appreciation to our project guide, Dr. M. Raghupathi, [Designation], Department of Computer Science and Engineering, and to our Course Coordinator, [Name of Course Coordinator], for their tremendous support, encouragement and guidance throughout this project.

We express our gratitude to **Dr. P. Venkateshwara Rao (CSE)**, Head of the ***Department of Computer Science Engineering***, for providing us with the facilities and the resources needed to complete this project-based lab.

We also thank K L (Deemed to be) University, the teaching and non-teaching staff of the department, and all those who helped us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

SANKEERTHANA THANMAYE	**2520090193**
VIKRAM IMMELLA	**2520090118**

Abstract

When several processes need data from a disk at the same time, the operating system must decide the order in which the pending requests are served. On a mechanical disk this order decides how far the read/write head has to travel, and therefore how much seek time is spent. Disk scheduling is normally taught with static diagrams and manual arithmetic, which makes it hard to follow and slow to compare. This project, the Disk Scheduling Algorithm Visualizer, addresses the problem with an interactive web application that simulates, animates and compares four classic algorithms: First Come First Served (FCFS), Shortest Seek Time First (SSTF), SCAN and C-SCAN.

The user enters the request queue, the initial head position, the minimum and maximum cylinder and the initial direction. The input is validated and every problem is reported with a specific message. The selected algorithm then produces the service sequence, the full head path, every individual seek distance, the total head movement and the average movement per request. The head path is shown as an animated SVG diagram with play, pause, step and speed controls, and all four algorithms can be compared on identical input with a table and a bar chart. A short Learn section summarises the algorithms. The counting convention used (the trip to the disk end and the C-SCAN wrap-around are counted) is stated in the interface.

The project is implemented in JavaScript with React and Vite, uses Recharts for the chart, and runs entirely in the browser without a backend. The algorithm logic is separated from the interface and is verified against textbook values by 14 automated unit tests, all of which pass. On the standard example queue with the head at 53, the totals are 640 (FCFS), 236 (SSTF), 331 (SCAN, right) and 382 (C-SCAN, right). The project demonstrates core Operating Systems concepts: I/O scheduling, request queues, seek time, starvation and fairness. It serves as an interactive learning tool for students and a demonstration tool for teachers.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		9–10
		1.1 Background of the Project	9
		1.2 Problem Statement	9
		1.3 Scope of the Project	9
		1.4 Objectives of the Project	10
		1.5 Importance of the Application	10
		1.6 Target Users and Audience	10
2	OS Concepts and Background Study		11–12
		2.1 Operating System Concepts Related to the Project	11
		2.2 Disk Structure: Platters, Tracks and Cylinders	11
		2.3 Disk Access Time	11
		2.4 Disk Scheduling	11
		2.5 First Come, First Served (FCFS)	11
		2.6 Shortest Seek Time First (SSTF)	11
		2.7 SCAN (Elevator Algorithm)	12
		2.8 Circular SCAN (C-SCAN)	12
		2.9 Existing Methods and Tools	12
		2.10 Proposed Approach	12
		2.11 Advantages of the Proposed Approach	12
3	System Requirements		13–14
		3.1 Hardware Requirements	13
		3.2 Software Requirements	13
		3.3 Operating System Environment	13
		3.4 Node.js and npm	13
		3.5 Development Tools	13
		3.6 Minimum System Requirements	14
		3.7 Functional Requirements	14
		3.8 Non-Functional Requirements	14
4	Technology and Tools Used		15–16
		4.1 JavaScript	15
		4.2 React	15

S. No.	Chapter	Topics / Subtopics	Page No.
		4.3 Vite	15
		4.4 HTML5 and CSS3	15
		4.5 SVG Graphics	15
		4.6 Recharts	15
		4.7 Lucide React	15
		4.8 Node Test Runner	15
		4.9 Visual Studio Code and Google Chrome	15
		4.10 Change from the Original Proposal	15
5	System Architecture and Design		17–22
		5.1 Overall System Architecture	17
		5.2 System Workflow	17
		5.3 Architecture Diagram	17
		5.4 Data Flow Diagram	17
		5.5 Simulation Flowchart	18
		5.6 Algorithm Flowcharts	19
		5.7 Module Description	20
		5.8 Input and Output Design	20
		5.9 Error Handling and Input Validation	21
		5.10 Design Decisions	21
6	Implementation		23–25
		6.1 Project Directory Structure	23
		6.2 Module-Wise Implementation	23
		6.3 The Common Result Builder	23
		6.4 FCFS Implementation	23
		6.5 SSTF Implementation	24
		6.6 SCAN Implementation	24
		6.7 C-SCAN Implementation	24
		6.8 Input Validation	24
		6.9 Animation Hook	25
		6.10 Visualization and Results Panel	25
		6.11 Comparison Module	25
		6.12 Algorithms and Pseudocode	25
		6.13 Build Process	25
7	Execution and Demonstration		26–30
		7.1 Environment Setup	26

S. No.	Chapter	Topics / Subtopics	Page No.
		7.2 Installation and Start-up	26
		7.3 Execution Commands	26
		7.4 Application Layout	26
		7.5 Running a Simulation	27
		7.6 Animation Controls	28
		7.7 Comparing the Algorithms	29
		7.8 Error Handling Demonstration	29
		7.9 Learn Section	30
		7.10 Demonstration of Operating Systems Concepts	30
8	Testing and Results		31–33
		8.1 Testing Methodology	31
		8.2 Unit Test Results	31
		8.3 Results on the Textbook Example	31
		8.4 Edge Cases	32
		8.5 Manual User-Interface Test Cases	32
		8.6 Results and Discussion	33
		8.7 Limitations	33
9	Conclusion and Future Enhancements		34
		9.1 Project Summary	34
		9.2 Objectives Achieved	34
		9.3 Operating Systems Learning Outcomes	34
		9.4 Benefits of the Application	34
		9.5 Current Limitations	34
		9.6 Future Enhancements	34
10	References		35
		10.1 Books	35
		10.2 Technology Documentation	35
		10.3 Course Material	35
		10.4 Project Files	35

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

An operating system (OS) manages the hardware of a computer and shares it among the running programs. One of its important duties is **input/output (I/O) management**. When several processes need data from a disk at the same time, the requests cannot all be served at once; they wait in a queue, and the OS must decide the order in which to serve them. This decision is called **disk scheduling**.

On a traditional mechanical hard disk, data is stored on circular tracks. All the tracks at the same distance from the centre of the platters form a **cylinder**, and a read/write head sits over one cylinder at a time. To reach a request on another cylinder the head must move, and this movement (the **seek**) is the slowest mechanical part of a disk access. Serving requests in a poor order makes the head travel back and forth unnecessarily and lowers the throughput of the whole system.

The **Disk Scheduling Algorithm Visualizer** is an interactive web application that simulates, animates and compares four classic disk scheduling algorithms: **FCFS** (First Come, First Served), **SSTF** (Shortest Seek Time First), **SCAN** and **C-SCAN** (Circular SCAN). It was built for the course Operating Systems and Systems Programming (OSSP) with React and Vite, and it runs entirely in the browser with no backend, database or login.

1.2 Problem Statement

The problem addressed by this project is: **"To implement and compare disk scheduling algorithms and show how the order of servicing disk requests affects disk-head movement."**

Disk scheduling is normally taught with a few hand-drawn diagrams and manual calculations. This approach has several difficulties:

- **Hard to follow:** the head path of an algorithm is drawn once on a board, so students cannot see how it develops move by move.
- **Error-prone arithmetic:** the total seek distance is the sum of many terms such as $|98 - 53|$, and a single slip changes the answer.
- **No quick comparison:** repeating the same exercise for four algorithms on the same queue takes a long time, so students rarely try different inputs.
- **Hidden conventions:** textbooks differ on whether the trip to the disk end in SCAN and the return jump in C-SCAN are counted, and this is seldom stated clearly.

The project therefore provides one tool in which the user enters a request queue, runs any of the four algorithms, watches the head move, sees every distance calculation, and compares all algorithms on identical input.

1.3 Scope of the Project

The scope of the project covers the following functions:

- Accepting a request queue, an initial head position, a minimum and maximum cylinder, an initial direction (left or right) and the algorithm to run.

- Validating all input and reporting clear messages for empty, malformed, out-of-range or inconsistent values (nothing is silently dropped).
- Simulating FCFS, SSTF, SCAN and C-SCAN and producing the service sequence, the full head path, every individual seek distance, the total head movement and the average movement per request.
- Animating the head path step by step with play, pause, next, previous, restart and speed controls.
- Comparing all four algorithms on the same input with a table and a bar chart, highlighting the lowest total movement.
- Providing a short "Learn Disk Scheduling" revision section inside the application.

The following are **outside** the scope of the current version and are listed as future work in Chapter 9: the LOOK and C-LOOK variants, rotational-latency and transfer-time modelling, arrival times for requests, simulation of solid-state drives, and saving or exporting results.

1.4 Objectives of the Project

1. To implement the FCFS, SSTF, SCAN and C-SCAN disk scheduling algorithms.
2. To accept the disk range, initial head position, request queue and scan direction as input and to validate them.
3. To display the service sequence of the disk head and calculate the total head movement and average seek distance.
4. To visualise the head movement as an animated diagram that can be stepped through.
5. To compare the algorithms using a results table and a chart on identical input.
6. To verify the algorithm logic with automated unit tests against known textbook values.

1.5 Importance of the Application

Disk scheduling is a core topic of the I/O management part of every Operating Systems course. The application turns a topic that is usually learned from static diagrams into one that can be explored: changing a single request or the starting direction immediately changes the path and the totals. Because the calculation of each seek distance is shown in full, the tool also helps students check their own manual answers.

1.6 Target Users and Audience

- **Students** of Operating Systems who want to understand and revise disk scheduling.
- **Teachers** who need a live demonstration for lectures and laboratory sessions.
- **Evaluators and reviewers** who want to verify the logic through the visible step-by-step calculations.

CHAPTER 2: OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

The project is based on the I/O management and secondary-storage management functions of an operating system. The main concepts are the disk request queue, the structure of a disk (tracks, cylinders and head), seek time, the disk scheduling algorithms and the criteria used to compare them: total head movement, throughput, fairness and starvation.

2.2 Disk Structure: Platters, Tracks and Cylinders

A hard disk consists of one or more circular platters coated with magnetic material. Each surface is divided into concentric **tracks**, and the tracks at the same radius on all surfaces together form a **cylinder**. A set of read/write heads moves together on an arm, so at any moment the heads are positioned over a single cylinder. In this project a request is simply the **cylinder number** that must be visited, and the disk is modelled as the range from a minimum cylinder (default 0) to a maximum cylinder (default 199).

2.3 Disk Access Time

The time needed to read or write data has three components:

- **Seek time:** the time to move the head to the required cylinder. It is roughly proportional to the distance moved.
- **Rotational latency:** the time for the required sector to rotate under the head.
- **Transfer time:** the time to actually move the data.

Seek time is usually the largest component on a mechanical disk and is the one the operating system can influence by choosing the order of requests. For this reason the project measures **head movement in cylinders** as the performance metric.

2.4 Disk Scheduling

Disk scheduling is the policy by which the OS orders the pending requests in the queue. A good policy reduces the total head movement, keeps the waiting time of every request bounded (no starvation) and treats requests fairly. The four algorithms implemented in this project are described in the following sections.

2.5 First Come, First Served (FCFS)

FCFS serves requests strictly in the order in which they arrive. It is the simplest policy and is fair because no request is starved, but it ignores the position of the head and can cause long zig-zag movements when requests are scattered across the disk.

2.6 Shortest Seek Time First (SSTF)

SSTF always selects the pending request closest to the current head position. It usually gives much less head movement than FCFS. However, it is a greedy policy: requests far from the head can wait for a very long time if new nearby requests keep arriving (**starvation**), and it does not guarantee the smallest possible total movement. In this project, ties are resolved deterministically in favour of the **lower cylinder** and then the earlier request.

2.7 SCAN (Elevator Algorithm)

SCAN moves the head in one direction, serving requests on the way, until it reaches the end of the disk. It then reverses and serves the remaining requests on the way back. Its behaviour resembles an elevator, so it is also called the elevator algorithm. It avoids starvation and has a bounded waiting time, but requests just behind the head must wait for a full sweep.

2.8 Circular SCAN (C-SCAN)

C-SCAN serves requests in one direction only. When the head reaches the disk end it jumps to the opposite end without serving anything and then continues in the **same** direction. This treats the cylinders as a circular list and gives a more uniform waiting time than SCAN, at the cost of the extra return movement.

2.9 Existing Methods and Tools

Disk scheduling is usually studied through textbook figures and hand calculation on a fixed example queue, such as the well-known queue 98, 183, 37, 122, 14, 124, 65, 67 with the head at cylinder 53. This is adequate for understanding the idea once, but it is static: the student cannot change the input, replay the movement or compare the algorithms quickly. Several textbook variants (LOOK and C-LOOK) and counting conventions also exist, which makes hand answers differ between sources.

2.10 Proposed Approach

The proposed approach is a browser-based simulator in which the algorithms are written as **pure JavaScript functions** that describe the head movement as a list of moves (service, boundary, wrap). A single function then derives all distances, totals and averages from those moves, so the mathematics exists in exactly one place. The user interface only displays the result object. The conventions used (for example, that the trip to the disk end and the C-SCAN wrap-around are counted) are stated openly in the interface.

2.11 Advantages of the Proposed Approach

- Immediate feedback: any change of input produces a new path and new totals.
- Step-by-step animation with the calculation of every seek distance visible.
- Side-by-side comparison of all algorithms on identical input.
- No installation of a server, database or API key; the application runs fully in the browser.
- Logic separated from the interface, which makes it easy to test automatically and to extend with new algorithms.

CHAPTER 3: SYSTEM REQUIREMENTS

3.1 Hardware Requirements

- A computer or laptop with a modern processor and at least 4 GB of RAM (8 GB recommended for comfortable development).
- About 500 MB of free disk space for the project folder and the installed dependencies (`node_modules`).
- A display of at least 1280 × 720 pixels; the layout is responsive and also adapts to smaller screens.

3.2 Software Requirements

Table 3.1: Software requirements

Software	Purpose
Node.js LTS (version 18 or newer) with npm	Runs the development server, installs packages and runs the tests
Visual Studio Code	Editing the source code and running the integrated terminal
Google Chrome (or another modern browser)	Running and demonstrating the application
Internet connection	Needed only once, for `npm install`; the application works offline afterwards

3.3 Operating System Environment

The application is developed and run on Windows with Visual Studio Code and Google Chrome, as documented in the project's README. Because the application consists only of JavaScript, HTML and CSS running in a browser, and Node.js is available for Linux as well, the same steps (`install`, `npm run dev`) apply on Ubuntu. [Confirm: was the project also run on Ubuntu/Linux? The project proposal mentions a Linux/Ubuntu environment. If it was tested there, state it here; otherwise keep this sentence as written.]

3.4 Node.js and npm

Node.js is a JavaScript runtime that executes JavaScript outside the browser. npm is its package manager. In this project Node.js is used by Vite (the development server and build tool) and by the built-in Node test runner (`node --test`), so no extra testing library is required.

3.5 Development Tools

- **Visual Studio Code** for editing, with its integrated terminal.
- **Google Chrome** with its developer tools for debugging the interface.
- **Vite** development server with hot reloading for fast feedback.

3.6 Minimum System Requirements

Any computer that can run a current browser and Node.js 18 or newer is sufficient. Once the production build has been created with `npm run build`, the generated `dist/` folder can be served from any static web host and needs only a browser on the user's side.

3.7 Functional Requirements

1. The system shall accept a list of cylinder requests, the initial head position, the minimum and maximum cylinder and the initial direction.
2. The system shall validate every input field and show a specific error message for each problem.
3. The system shall simulate FCFS, SSTF, SCAN and C-SCAN.
4. The system shall display the service sequence, the head path, every seek distance, the total movement and the average movement.
5. The system shall animate the head path with play, pause, next, previous, restart and speed controls.
6. The system shall compare all four algorithms on the same input in a table and a chart.
7. The system shall provide buttons to load the example input and to reset everything.

3.8 Non-Functional Requirements

- **Correctness:** results must match known textbook values (verified by automated tests).
- **Usability:** clear labels, helpful validation messages and a responsive layout.
- **Accessibility:** keyboard-friendly controls, visible focus states and error messages announced to assistive technology.
- **Performance:** simulations run instantly for the supported maximum of 60 requests.
- **Maintainability:** algorithm logic is separate from the user interface and each algorithm lives in its own file.
- **Portability:** runs in any modern browser without a backend.

CHAPTER 4: TECHNOLOGY AND TOOLS USED

4.1 JavaScript

All the logic of the project is written in JavaScript using ES modules. The four scheduling algorithms, the input validation and the animation state are plain JavaScript and do not depend on the user-interface library, which makes them easy to test.

4.2 React

React (version 18) is used to build the user interface from small reusable components such as the input form, the disk diagram, the results panel and the comparison table. The state of the page (form values, errors, current result, comparison and animation speed) is held in the `App` component and passed down to the components.

4.3 Vite

Vite (version 5) is the development server and build tool. `npm run dev` starts a local server, normally at `http://localhost:5173/`, with instant reloading; `npm run build` creates an optimised production version in the `dist/` folder.

4.4 HTML5 and CSS3

HTML5 provides the page structure and CSS3 provides the styling in a single stylesheet (`index.css`). The design is a responsive light theme with visible keyboard-focus states.

4.5 SVG Graphics

The animated diagram of the disk head is drawn as an SVG image generated by the `DiskVisualization` component. Normal service moves, moves to the disk boundary and C-SCAN wrap-around jumps have different colours and arrowheads, and a legend explains them.

4.6 Recharts

Recharts (version 2) is the charting library used for the comparison bar chart of total head movement for the four algorithms.

4.7 Lucide React

Lucide React provides the small icons used on buttons and headings.

4.8 Node Test Runner

The unit tests use the test runner built into Node.js (`node:test`) together with `node:assert`, so no separate testing framework is installed. The tests are run with `npm test`.

4.9 Visual Studio Code and Google Chrome

Visual Studio Code was used to write the code and Google Chrome was used to run and demonstrate the application.

4.10 Change from the Original Proposal

The project proposal planned the application with plain HTML, CSS, JavaScript and Chart.js. During development the team chose **React with Vite** and **Recharts** instead, because components

and state handling make the animated diagram, the validation messages and the comparison views easier to build and maintain. The functionality promised in the proposal (input of head position, disk size, request queue and direction; service sequence; total and average movement; chart and comparison table) is fully provided. The proposal also lists C/C++ as “not used”, and no C or C++ code is part of this implementation.

CHAPTER 5: SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The application follows a **layered, modular architecture**. The presentation layer consists of React components that display data and collect input. Below it is the App component, which holds the application state. The logic layer contains pure JavaScript modules: the validation module, the algorithm modules with their shared result builder, and the animation hook. The result of every simulation is one standard result object that the components render. The architecture is shown in Figure 5.1.

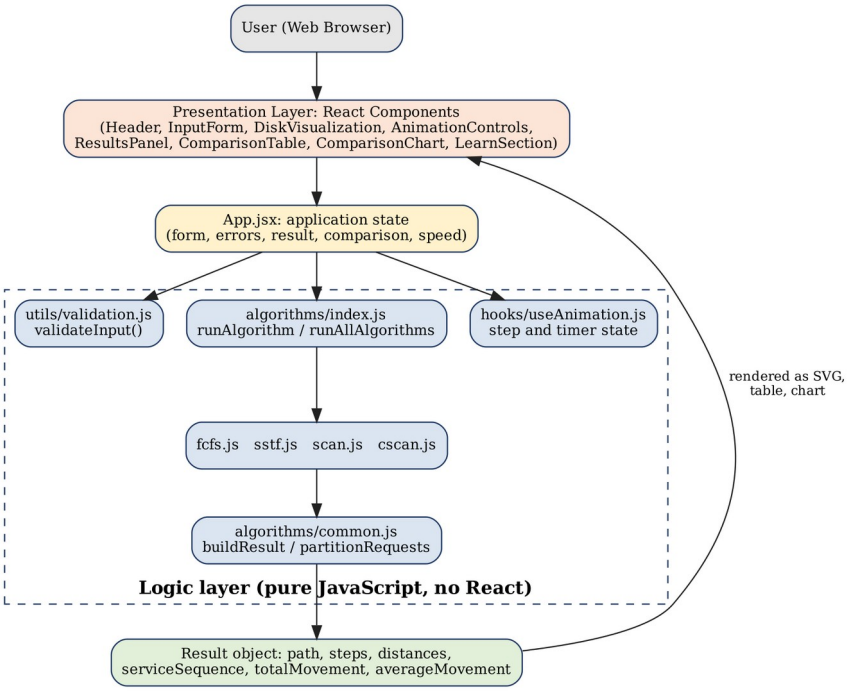


Figure 5.1: Overall system architecture of the Disk Scheduling Visualizer

5.2 System Workflow

The user enters or loads an input and chooses an algorithm. When **Run Simulation** is pressed, the form text is validated; if it is valid, the selected algorithm is executed and the head path, results and animation controls are shown. When **Compare All** is pressed, all four algorithms are executed on the same validated input and the comparison table and chart are shown. If the input is edited afterwards, the interface marks the existing results as out of date instead of silently showing wrong values.

5.3 Architecture Diagram

Figure 5.1 is the architecture diagram. The orange box is the presentation layer, the yellow box is the application state, the dashed region is the logic layer that contains no React code, and the green box is the result object shared by all algorithms. Data flows downwards as input and upwards as results.

5.4 Data Flow Diagram

Figure 5.2 shows the data flow. The user supplies the queue, head, range and direction to process 1.0, which validates them and either returns error messages or stores the valid values. Process 2.0

runs the selected algorithm (or all four), process 3.0 builds the result object with all distances and totals, and process 4.0 displays the SVG path, table and chart and responds to the animation controls.

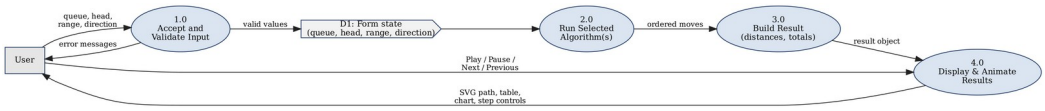


Figure 5.2: Data flow diagram of the visualizer

5.5 Simulation Flowchart

Figure 5.3 shows the steps performed when the user presses Run Simulation or Compare All. No algorithm is executed unless the validation succeeds.

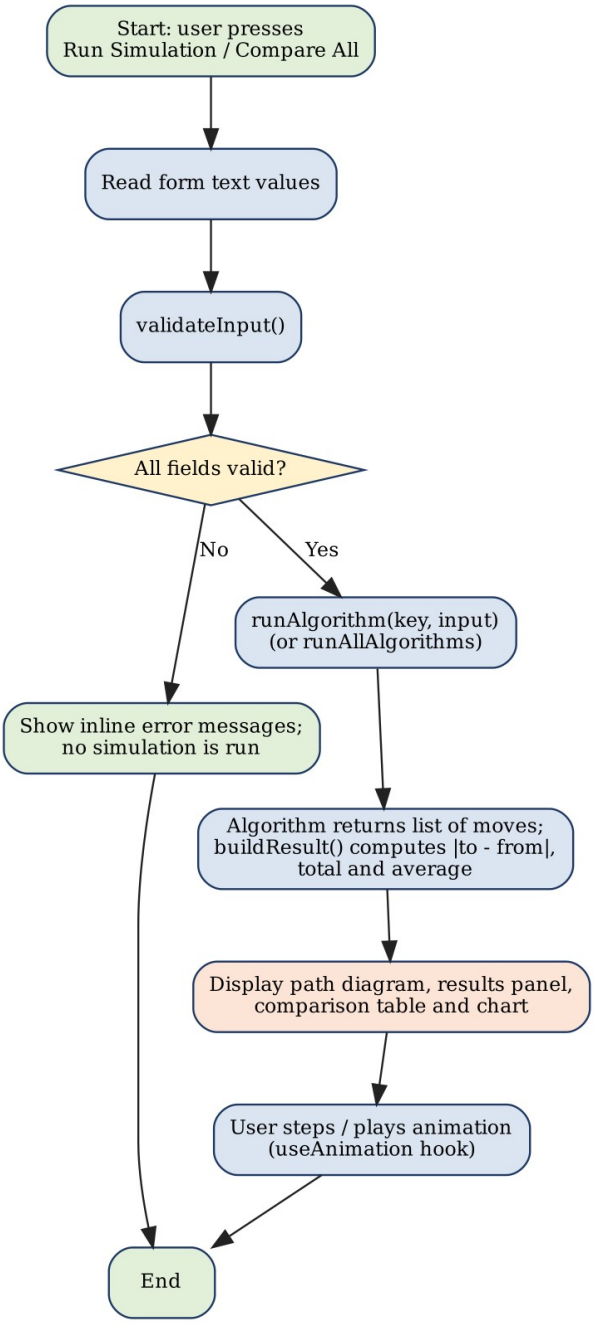


Figure 5.3: Flowchart of running a simulation

5.6 Algorithm Flowcharts

Figure 5.4 shows the logic of SSTF and Figure 5.5 shows the shared logic of SCAN and C-SCAN. FCFS needs no flowchart because it visits the requests in the input order.

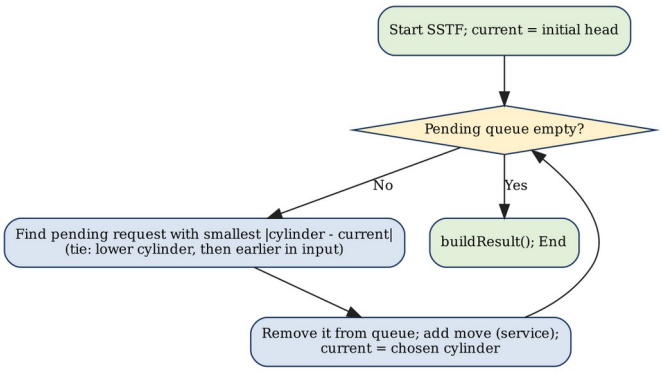


Figure 5.4: Flowchart of SSTF

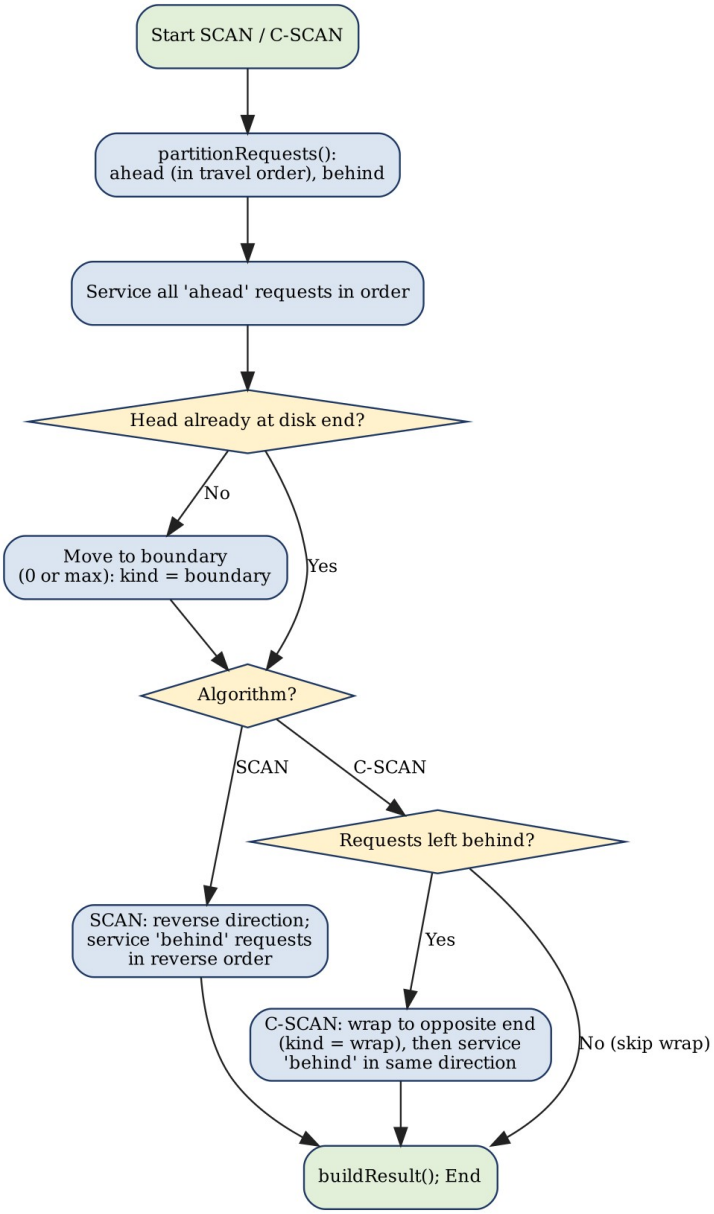


Figure 5.5: Flowchart of SCAN and C-SCAN

5.7 Module Description

Table 5.1: Modules and their responsibilities

Module / File	Responsibility	Main interface
`App.jsx`	Holds application state; connects the form, algorithms, animation and result views	React `useState`
`InputForm.jsx`	Collects queue, head, range, direction and algorithm; shows the four buttons and field errors	HTML form
`utils/validation.js`	Converts the form text to numbers and produces specific error messages	`validateInput()`
`algorithms/fcfs.js`, `sstf.js`, `scan.js`, `cscan.js`	Each describes the head movement of one algorithm as a list of moves	`fcfs()`, `sstf()`, `scan()`, `cscan()`
`algorithms/common.js`	Builds the result object; computes every distance, total and average; splits requests around the head	`buildResult()`, `partitionRequests()`
`algorithms/index.js`	Registry of the four algorithms; runs one or all	`runAlgorithm()`, `runAllAlgorithms()`
`hooks/useAnimation.js`	Step and play state with a single cleaned-up timer	React hook
`DiskVisualization.jsx`	Draws the animated SVG head path and legend	SVG
`AnimationControls.jsx`	Play, pause, next, previous, restart and speed selector	HTML buttons
`ResultsPanel.jsx`	Shows totals, service order, path and every distance calculation	HTML table/text
`ComparisonTable.jsx`, `ComparisonChart.jsx`	Compare all algorithms in a table and a bar chart	Recharts
`LearnSection.jsx`	Revision guide for disk scheduling and the four algorithms	Static content

5.8 Input and Output Design

Input comes from the keyboard and mouse: a request queue typed as numbers separated by commas, spaces or semicolons; the head position; the minimum and maximum cylinder; the direction; and the algorithm. **Output** consists of the animated diagram, the results panel and, for comparison, a table and a chart. Table 5.2 summarises the design.

Table 5.2: Input and output design

Item	Type	Validation / format
Request queue	List of integers	Not empty; whole numbers only; at most 60 requests; every value within the cylinder range

Item	Type	Validation / format
Initial head	Integer	Required; whole number within the cylinder range
Min / Max cylinder	Integers	Required; whole numbers; maximum must be greater than minimum
Direction	Left / Right	Used by SCAN and C-SCAN
Algorithm	FCFS / SSTF / SCAN / C-SCAN	Chosen with a selector
Head path diagram	SVG	Service, boundary and wrap moves in different colours
Results	Text and tables	Total, average, service order, path, each `|to − from| = d`
Comparison	Table and bar chart	Lowest total movement highlighted

5.9 Error Handling and Input Validation

Because all input is typed text, the validation module checks each field and returns a friendly message instead of throwing an error. Nothing is silently dropped or corrected. Table 5.3 lists the situations and the responses.

Table 5.3: Validation situations and responses

Situation	Response
Empty request queue	"Enter at least one cylinder request, separated by commas."
Non-integer value in the queue (for example `abc`, `12.5`)	"Invalid value(s): …. Use whole numbers separated by commas." The offending values are listed
More than 60 requests	"Too many requests (n). Please use at most 60."
Request outside the range	"Out of range (min–max): …" with the offending values
Head empty, not a whole number or outside the range	Message on the head field
Min or max empty or not a whole number	Message on that field
Minimum not smaller than maximum	"Maximum cylinder must be greater than the minimum cylinder."
Inputs changed after a run	A notice that the displayed result or comparison is out of date

5.10 Design Decisions

- **Single place for the mathematics:** every algorithm only returns a list of moves of type service, boundary or wrap; `buildResult()` computes all distances, totals and averages.
- **Counting convention:** the trip to the disk end (SCAN, C-SCAN) and the C-SCAN wrap-around jump are part of the head path and are **included** in the total movement. The average is the total divided by the number of real requests. This is stated in the interface because some textbooks do not count the return jump.

- **No pointless wrap:** if no requests remain after the C-SCAN head reaches the boundary, the wrap-around is skipped.
- **Deterministic SSTF:** ties are resolved to the lower cylinder, then to the earlier request.
- **Physical disk ends:** SCAN and C-SCAN go to the disk end (minimum or maximum cylinder), not just to the last request; LOOK and C-LOOK are not implemented.

CHAPTER 6: IMPLEMENTATION

6.1 Project Directory Structure

```
disk-scheduling-visualizer/
├── package.json   vite.config.js   index.html   README.md
├── tests/
│   └── algorithms.test.js
└── src/
    ├── main.jsx   App.jsx   index.css
    ├── algorithms/   common.js  fcfs.js  sstf.js  scan.js  cscan.js  index.js
    ├── hooks/        useAnimation.js
    ├── utils/        validation.js
    └── components/   Header  InputForm  DiskVisualization  AnimationControls
                      ResultsPanel  ComparisonTable  ComparisonChart
                      LearnSection
```

6.2 Module-Wise Implementation

The implementation follows the modules described in Table 5.1. The most important design rule is that the folder `src/algorithms/` contains pure logic with no React code, while the React components only display the result object.

6.3 The Common Result Builder

Every algorithm describes its head movement as a list of moves of the form `{ to, kind }`, where `kind` is `service` (a request is reached), `boundary` (SCAN/C-SCAN travel to the disk end) or `wrap` (C-SCAN jump to the opposite end). The function `buildResult()` in `common.js` walks through these moves, computes `|to − from|` for each, and returns the standard result object with the fields `serviceSequence`, `path`, `steps`, `distances`, `totalMovement`, `averageMovement` and `requestsServiced`.

```
let current = initialHead;
const steps = moves.map((move, i) => {
  const distance = Math.abs(move.to - current);
  const step = { index: i + 1, from: current, to: move.to, distance, kind:
move.kind };
  current = move.to;
  return step;
});
const totalMovement = steps.map(s => s.distance).reduce((sum, d) => sum + d, 0);
// averageMovement = totalMovement / number of requests serviced
```

6.4 FCFS Implementation

FCFS simply maps the request list, in input order, to a list of service moves.

```
export function fcfs(input) {
  const moves = input.requests.map((cylinder) => ({ to: cylinder, kind:
STEP_KIND.SERVICE }));
  return buildResult({ key: 'fcfs', algorithm: 'FCFS', input, moves });
}
```

6.5 SSTF Implementation

SSTF keeps a pending list. At each step it finds the request with the smallest distance to the current head (lower cylinder wins a tie, then the earlier request), removes it from the list and adds a service move.

```
while (pending.length > 0) {
  let bestIndex = 0;
  for (let i = 1; i < pending.length; i++) {
    const dCandidate = Math.abs(pending[i].cylinder - current);
    const dBest = Math.abs(pending[bestIndex].cylinder - current);
    const closer = dCandidate < dBest;
    const tieLowerCylinder = dCandidate === dBest && pending[i].cylinder <
pending[bestIndex].cylinder;
    // (equal cylinders: the earlier request in the input wins)
    if (closer || tieLowerCylinder /* || tieEarlier */) bestIndex = i;
  }
  const [chosen] = pending.splice(bestIndex, 1);
  moves.push({ to: chosen.cylinder, kind: STEP_KIND.SERVICE });
  current = chosen.cylinder;
}
```

6.6 SCAN Implementation

`partitionRequests()` splits the requests into those **ahead** of the head in the chosen direction (already in travel order) and those **behind**. SCAN services the requests ahead, travels to the disk boundary unless the head is already there, and then services the requests behind in the reverse direction.

```
const { ahead, behind } = partitionRequests(requests, initialHead, direction);
const returning = goingRight ? [...behind].reverse() : behind;
ahead.forEach((c) => moves.push({ to: c, kind: STEP_KIND.SERVICE }));
const reached = ahead.length > 0 ? ahead[ahead.length - 1] : initialHead;
if (reached !== boundary) moves.push({ to: boundary, kind: STEP_KIND.BOUNDARY });
returning.forEach((c) => moves.push({ to: c, kind: STEP_KIND.SERVICE }));
```

6.7 C-SCAN Implementation

C-SCAN is identical up to the boundary. Then, if requests remain behind the head, it adds a **wrap** move to the opposite end and services the remaining requests in the same direction.

```
const afterWrap = goingRight ? behind : [...behind].reverse();
...
if (reached !== boundary) moves.push({ to: boundary, kind: STEP_KIND.BOUNDARY });
if (afterWrap.length > 0) {
  moves.push({ to: wrapTarget, kind: STEP_KIND.WRAP });
  afterWrap.forEach((c) => moves.push({ to: c, kind: STEP_KIND.SERVICE }));
}
```

6.8 Input Validation

`validateInput()` in `utils/validation.js` receives the form text, parses each integer with a strict pattern (`/^[+-]?\d+$/`), checks the range, head and queue, and returns `{ valid, errors, values }`. When the input is valid, `values` contains the numbers ready for the algorithms; otherwise `errors` holds one message per faulty field.

6.9 Animation Hook

The hook `useAnimation(totalSteps, resetKey, speedMs)` keeps the current step index (0 is the initial head position, k means the first k moves are complete) and a playing flag. Only one timer exists at any time: it is created by an effect while playing and always removed in the effect cleanup, so pausing, restarting, running a new simulation, changing the speed or leaving the page cannot leave a stray timer running. Playback stops automatically at the last step and replays from the start if Play is pressed again.

6.10 Visualization and Results Panel

`DiskVisualization` converts the path into SVG coordinates and draws an arrow for each completed move. Boundary moves and wrap moves use different colours (wrap is dashed). `ResultsPanel` lists the service order, the full path, each calculation in the form `|to − from| = d`, and the total as a sum, and marks results that no longer match the form.

6.11 Comparison Module

When Compare All is pressed, `runAllAlgorithms()` executes all four algorithms on the same validated input. `ComparisonTable` shows total movement, average movement and the service order for each, and highlights the lowest total; `ComparisonChart` shows the totals as a bar chart.

6.12 Algorithms and Pseudocode

```
FCFS(requests, head):
    for each r in requests (input order): move head to r

SSTF(requests, head):
    pending = requests
    while pending not empty:
        r = request in pending with minimum |r - head|   (tie: lower cylinder)
        move head to r; remove r from pending

SCAN(requests, head, direction, min, max):
    ahead  = requests in travel direction, nearest first
    behind = requests on the other side
    serve ahead; move to boundary (max if right, min if left) unless already there
    reverse direction; serve behind nearest-first

C-SCAN(requests, head, direction, min, max):
    serve ahead; move to boundary unless already there
    if behind not empty:
        jump to opposite end            (counted in total movement)
        serve behind in the SAME direction
```

6.13 Build Process

The project is built and run with Vite through the scripts defined in `package.json`: `npm run dev` (development server), `npm run build` (production build into `dist/`), `npm run preview` (local preview of the production build, normally at `http://localhost:4173/`) and `npm test` (unit tests).

CHAPTER 7: EXECUTION AND DEMONSTRATION

Note to authors (delete before submission): the head-path figures in this chapter were generated from the project's own algorithm code for the example input. Replace the yellow screenshot boxes with real screenshots of your running application.

7.1 Environment Setup

1. Install Node.js LTS (version 18 or newer) from nodejs.org and check it with `node -v` and `npm -v`.
2. Install Visual Studio Code and Google Chrome.
3. Extract the project ZIP and open the folder that contains `package.json` in Visual Studio Code (File → Open Folder).
4. Open the integrated terminal (Terminal → New Terminal).

7.2 Installation and Start-up

Install the dependencies once (this is the only step that needs the internet):

```
npm install
```

Start the development server:

```
npm run dev
```

The terminal prints a local address, usually `http://localhost:5173/`, which is opened in Google Chrome. The server is stopped with Ctrl+C. If PowerShell reports that running scripts is disabled, the terminal can be switched to Command Prompt and the same commands used.

7.3 Execution Commands

Table 7.1: Commands used

Command	Purpose
`npm install`	Install React, Vite, Recharts, Lucide React and the plugin
`npm run dev`	Start the development server (normally port 5173)
`npm run build`	Create the optimised production version in `dist/`
`npm run preview`	Preview the production build locally (normally port 4173)
`npm test`	Run the 14 unit tests with the Node test runner

7.4 Application Layout

The page has a header, a hero section with the project title, the **Simulator** section, the **Algorithm comparison** section and the **Learn Disk Scheduling** section. In the Simulator section the input form is on the left and the animated disk diagram is on the right (stacked on narrow screens), with the results panel below them.

[Insert screenshot 7.1: Full application page with the example input loaded]

Screenshot 7.1: Full application page with the example input loaded

7.5 Running a Simulation

Pressing **Load Example** fills the form with the queue `98, 183, 37, 122, 14, 124, 65, 67`, head `53`, range `0–199` and direction `right`. Selecting an algorithm and pressing **Run Simulation** produces the head path. Figures 7.1 to 7.4 show the head paths for the four algorithms on this input (direction right) as produced by the project's algorithm code; the application draws the same paths in its SVG diagram.

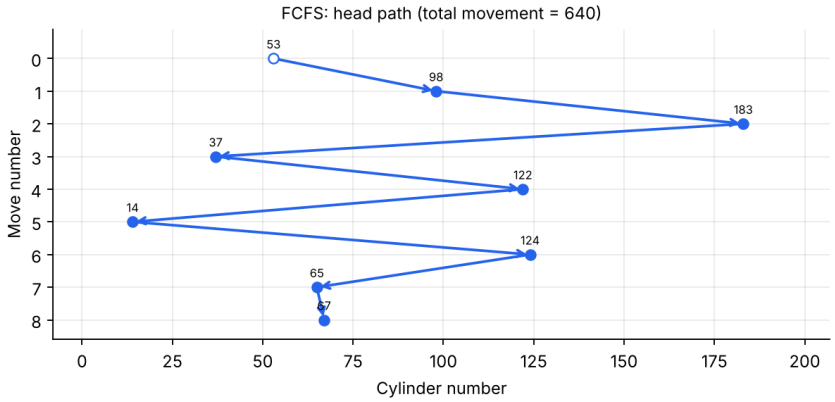


Figure 7.1: FCFS head path (total movement 640)

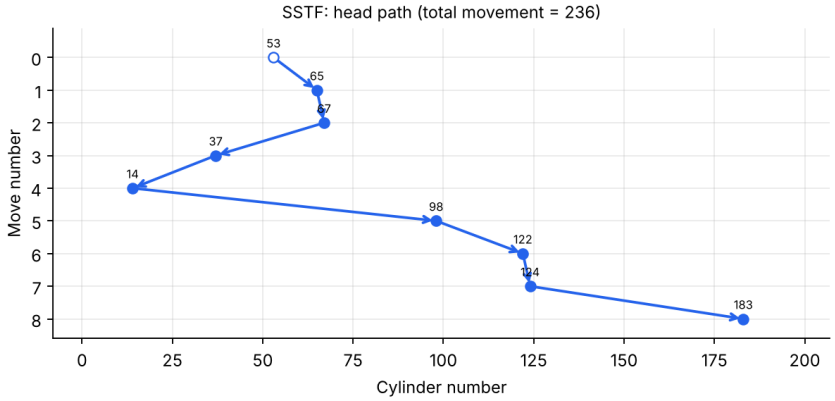


Figure 7.2: SSTF head path (total movement 236)

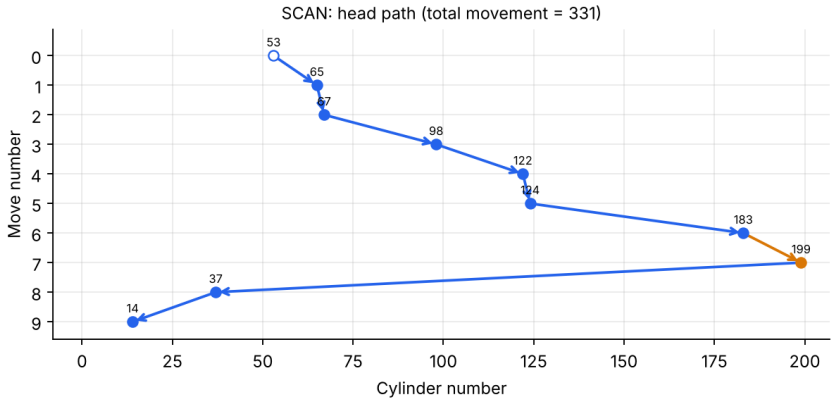


Figure 7.3: SCAN head path, direction right (total movement 331)

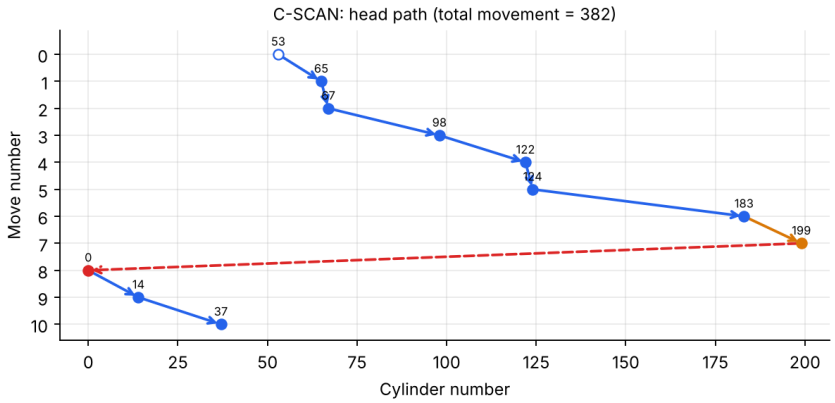


Figure 7.4: C-SCAN head path, direction right (total movement 382)

In the figures, blue arrows are service moves, the orange arrow is the trip to the disk boundary, and the dashed red arrow is the C-SCAN wrap-around jump, as in the application's legend.

7.6 Animation Controls

Below the diagram are the buttons **Play**, **Pause**, **Next Step**, **Previous Step** and **Restart Animation**, and a speed selector. A new simulation always starts paused at the initial head position. The results panel follows the animation so that the calculation of the move just completed is visible.

[Insert screenshot 7.2: Animation partly completed (SCAN, direction right) with the controls visible]

Screenshot 7.2: Animation partly completed (SCAN, direction right) with the controls visible

7.7 Comparing the Algorithms

Pressing **Compare All** runs all four algorithms on the same input and shows the table and bar chart (Figure 7.5 shows the totals). The algorithm with the lowest total is highlighted.

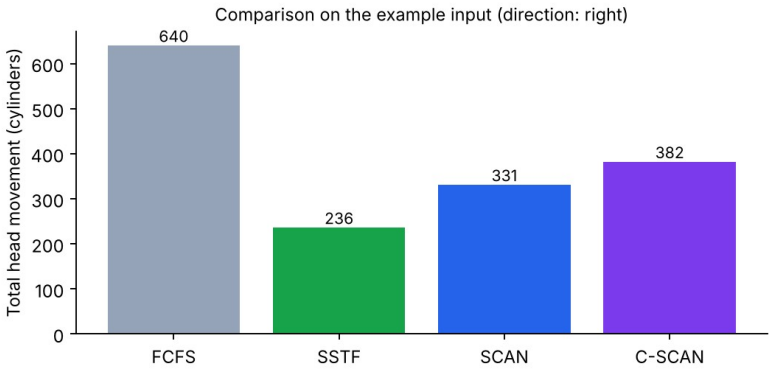


Figure 7.5: Total head movement of the four algorithms on the example input

[Insert screenshot 7.3: Comparison table and chart in the application]

Screenshot 7.3: Comparison table and chart in the application

7.8 Error Handling Demonstration

The validation messages can be demonstrated with the following inputs:

Table 7.2: Validation demonstration

Input	Message shown
Empty queue	Enter at least one cylinder request, separated by commas.
Queue `10, abc, 12.5`	Invalid values: abc, 12.5. Use whole numbers separated by commas.
Queue `10, 250, -3` with range 0–199	Out of range (0–199): 250, -3.
Head `500` with range 0–199	Initial head must be between 0 and 199.
Min `200`, Max `199`	Maximum cylinder must be greater than the minimum cylinder.

[Insert screenshot 7.4: Validation errors shown on the input form]

Screenshot 7.4: Validation errors shown on the input form

7.9 Learn Section

The Learn Disk Scheduling section gives five short explanations (what disk scheduling is, cylinder and head, why seek distance matters, FCFS vs SSTF, SCAN vs C-SCAN) and a card for each algorithm with how it works, its advantage, its drawback and its behaviour.

[Insert screenshot 7.5: Learn Disk Scheduling section]

Screenshot 7.5: Learn Disk Scheduling section

7.10 Demonstration of Operating Systems Concepts

- **Request queue and I/O scheduling:** the input queue and the different service orders.
- **Seek time and head movement:** every `|to − from| = d` calculation and the totals.
- **Starvation and fairness:** discussed in the Learn section for SSTF versus SCAN/C-SCAN.
- **Policy comparison:** the same queue yields different totals under different policies.

CHAPTER 8: TESTING AND RESULTS

8.1 Testing Methodology

Testing combined three methods: (1) **automated unit tests** of the algorithms and the validation module, run with `npm test`; (2) **checking the results against hand calculation** of the well-known textbook example; and (3) **manual testing of the user interface** in the browser. The algorithm logic is independent of React, so it could be tested directly without a browser.

8.2 Unit Test Results

The test file `tests/algorithms.test.js` contains 14 tests. When run with `npm test`, all **14 tests passed and none failed**. Table 8.1 lists them.

Table 8.1: Automated unit tests

No.	Test	What it checks
1	FCFS on the textbook example	Sequence, 8 distances, total 640, average 80
2	SSTF on the textbook example	Sequence 65, 67, 37, 14, 98, 122, 124, 183; total 236
3	SCAN going right	Path reaches 199 then reverses; total 331
4	SCAN going left	Path reaches 0 then reverses; total 236
5	C-SCAN going right	Wraps 199 → 0; total 382
6	C-SCAN going left	Wraps 0 → 199; total 386
7	SSTF tie-breaking	Equal distances go to the lower cylinder
8	C-SCAN with nothing behind the head	Wrap-around is skipped; total 189
9	Head already on the boundary	No zero-length boundary step is added
10	Request at the head and duplicate requests	All requests are serviced by every algorithm
11	Single request	Works for every algorithm and direction
12	Random inputs (300 rounds × 2 directions × 4 algorithms)	Path starts at head and stays in range; totals equal the sum of distances; all requests serviced exactly once; average = total / count
13	Validation accepts the example	Valid input is converted to numbers
14	Validation rejects bad input	Empty queue, bad numbers, out-of-range values, bad head and bad range are reported

8.3 Results on the Textbook Example

For the queue 98, 183, 37, 122, 14, 124, 65, 67 with the head at 53 and the range 0–199, the results of the project are shown in Table 8.2. They were obtained from the project's algorithm code and agree with hand calculation of the individual distances.

Table 8.2: Results on the example input

Algorithm	Total movement	Average / request	Requests
FCFS	640	80.00	8
SSTF	236	29.50	8
SCAN (right)	331	41.38	8
SCAN (left)	236	29.50	8
C-SCAN (right)	382	47.75	8
C-SCAN (left)	386	48.25	8

Hand verification for two cases: **FCFS** distances are 45 + 85 + 146 + 85 + 108 + 110 + 59 + 2 = 640. **C-SCAN (right)** is 146 (53 → 199) + 199 (wrap to 0) + 37 (0 → 37) = 382, because the requests 14 and 37 are serviced after the wrap.

8.4 Edge Cases

- A request exactly at the head position counts as “ahead” with distance 0.
- Duplicate cylinders are serviced individually and in input order.
- If the head already sits on the disk end, no zero-length boundary move is added.
- C-SCAN skips the wrap-around when no request remains behind the head.
- A queue with one request works for all algorithms and both directions.

8.5 Manual User-Interface Test Cases

Note to authors (delete before submission): these interface tests could not be run by the report writer because the application must be run in a browser. Run each one and write Pass or Fail in the last column.

Table 8.3: Manual test cases

No.	Test	Expected result	Actual result
1	Press Load Example, select FCFS, press Run Simulation	Path and total 640 shown	[Pass / Fail]
2	Press Compare All	Table and chart show 640, 236, 331, 382; lowest highlighted	[Pass / Fail]
3	Run SCAN and press Play	Head animates step by step and stops at the last step	[Pass / Fail]
4	Press Pause, Next Step and Previous Step	Animation pauses; steps move one at a time	[Pass / Fail]
5	Press Restart Animation	Returns to the initial head position	[Pass / Fail]
6	Change direction to left and run C-SCAN	Total 386 with a wrap from 0 to 199	[Pass / Fail]
7	Enter an invalid queue (`10, abc`)	Specific error message; no simulation is run	[Pass / Fail]
8	Edit the queue after a run	Notice that the result is out of date	[Pass / Fail]

No.	Test	Expected result	Actual result
9	Press Reset	Form returns to defaults; results are cleared	[Pass / Fail]
10	Resize the browser window to a phone width	Layout stacks and stays usable	[Pass / Fail]

8.6 Results and Discussion

On the example input, FCFS has by far the largest movement (640) because it follows the arrival order and zig-zags across the disk. SSTF gives 236, and SCAN gives 331 or 236 depending on the starting direction. C-SCAN gives 382 (right) and 386 (left); its totals are higher here because the wrap-around jump is counted, in exchange for a more uniform waiting time. Which algorithm has the lowest total depends on the queue, the head position and the direction, so the application lets the user try other inputs. A lower total movement is also not automatically “best”: SSTF can starve distant requests, and SCAN and C-SCAN bound the waiting time.

8.7 Limitations

- The model counts head movement in cylinders; rotational latency and transfer time are not modelled.
- All requests are assumed to be present at the start; arrival times are not simulated.
- SCAN and C-SCAN always go to the physical disk end; LOOK and C-LOOK are not implemented.
- The model describes traditional mechanical disks; SSDs and drives with command queueing behave differently.
- The queue is limited to 60 requests.

CHAPTER 9: CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Project Summary

The Disk Scheduling Algorithm Visualizer is a browser-based application that implements FCFS, SSTF, SCAN and C-SCAN, validates user input, animates the disk-head path step by step, shows every seek-distance calculation, and compares the algorithms with a table and a chart. The algorithm logic is separate from the interface and is verified by 14 automated unit tests, all of which pass.

9.2 Objectives Achieved

- The four disk scheduling algorithms were implemented.
- Disk range, head position, request queue and direction are accepted and validated.
- The service sequence, total head movement and average seek distance are displayed.
- The head movement is animated and can be stepped through.
- The algorithms are compared with a chart and a table.
- The logic is verified against known textbook values.

9.3 Operating Systems Learning Outcomes

- Understanding how an OS orders disk requests and why seek distance matters.
- Understanding the strengths and weaknesses of each algorithm: fairness, starvation and waiting time.
- Understanding that counting conventions (the boundary trip and the wrap-around) affect the numbers and must be stated.
- Practising separation of logic and interface, input validation and automated testing.

9.4 Benefits of the Application

The application gives students an interactive way to learn and revise disk scheduling, gives teachers a ready demonstration tool, and requires no server, database or account.

9.5 Current Limitations

The limitations of the current version are those listed in Section 8.7: only head movement in cylinders is modelled, all requests arrive at the start, LOOK and C-LOOK are not included, and the model assumes a mechanical disk.

9.6 Future Enhancements

- Add the LOOK and C-LOOK variants and let the user choose whether the return jump is counted.
- Model rotational latency and transfer time to estimate total access time.
- Allow requests with arrival times and compute waiting time and starvation measures.
- Add an option to export results or the diagram as an image or PDF.
- Add a comparison against solid-state drive behaviour.

CHAPTER 10: REFERENCES

10.1 Books

1. A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, Wiley (chapters on mass-storage structure and disk scheduling).
2. A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, Pearson (chapter on input/output and disk arm scheduling).
3. W. Stallings, *Operating Systems: Internals and Design Principles*, Pearson (chapter on I/O management and disk scheduling).

10.2 Technology Documentation

1. React documentation, https://react.dev
2. Vite documentation, https://vite.dev
3. Recharts documentation, https://recharts.org
4. Lucide icons documentation, https://lucide.dev
5. Node.js documentation: test runner (`node:test`) and assert, https://nodejs.org/docs
6. MDN Web Docs: SVG, HTML and CSS reference, https://developer.mozilla.org

10.3 Course Material

1. Operating Systems and Systems Programming (25CS2104E), K L (Deemed to be) University, Term-I 2026–27: course notes and project problem-statement form.

10.4 Project Files

1. Source code and README of the project “disk-scheduling-visualizer”. GitHub repository: [add your repository link]. Deployed website: [add your website link].